

NovaSMS

Frontend, Modèle de Données & Parcours Utilisateurs

Pages & features React · Architecture applicative · Relations clés du modèle de données · Parcours utilisateur bout-en-bout

Stage Sankofa Lab · Romuald KO · Document régénéré depuis le code réel du dépôt
Plateforme SaaS B2B Messagerie Multi-Canal — NestJS · PostgreSQL · Redis · React 19

Table des matières

1	Vue d'ensemble technique frontend
2	Architecture applicative — routing, état, couche API
3	Authentification — pages et flux
4	Dashboard
5	Campagnes — liste, wizard, éditeurs, rapport
6	Contacts — liste, import, détail, segments
7	Automations
8	Analytics
9	Rechargement / Billing
10	Compte / Paramètres
11	Modèle de données — relations clés côté produit
12	Parcours utilisateur — inscription au premier envoi
13	Parcours utilisateur — recharge de crédit Mobile Money
14	Incohérences et dette technique frontend identifiées

1 — Vue d'ensemble technique frontend

Le frontend est une SPA React 19.2 servie par Vite 8, écrite en TypeScript. Le point d'entrée réel est src/main.tsx (routing + code-splitting par React.lazy + gestion globale des erreurs) — le fichier src/App.tsx, historiquement le point d'entrée conventionnel d'un projet Vite/React, est un vestige qui retourne simplement null et n'est importé nulle part.

Domaine	Choix technique	Constat d'usage réel
Routing	react-router-dom 7, BrowserRouter classique	Pas de data router, pas de loaders/actions
État global	Zustand 5 + persist	3 stores actifs : authStore, uiStore, campaign.store
Données serveur	axios + useEffect manuel	Pas de React Query/SWR — fetch dupliqué page par page
Style	Tailwind 3.4 + CSS custom (index.css, 728 lignes)	Pas de design system de composants réutilisables
Formulaires	react-hook-form + zod	Utilisé uniquement sur l'inscription ; le reste en useState manuel
i18n	i18next initialisé globalement	Utilisé dans un seul composant (Header) sur 2 clés

2.1 Routing et protection des routes

Toutes les pages protégées passent par un wrapper commun `<ProtectedRoute><AppLayout /></ProtectedRoute>` avec un `<Outlet />`, chaque route enfant pouvant être enveloppée d'un `<RoleGuard roles={[...]}>`. `ProtectedRoute` attend l'hydratation du store Zustand persisté (`isHydrated`) avant de statuer, pour éviter une redirection prématurée vers `/login` au premier rendu.

2.2 Gestion d'état — pourquoi Zustand et comment il est utilisé

- **authStore (persist, clé novasms-auth)** !' accessToken, refreshToken, user, expiration de session (8h normal / 30 jours "se souvenir de moi")
- **uiStore (persist, clé novasms-ui)** !' préférences d'affichage (ex. dashboard actif 1 ou 2)
- **campaign.store (persist)** !' brouillon de campagne en cours d'édition, survit à un rafraîchissement de page
- **Pas de store pour contacts/segments/automations** !' chaque page gère son propre `useState` local et un `key={refreshKey}` pour forcer un `refetch` après mutation

2.3 Couche API et intercepteur Axios

`src/api/axios.ts` centralise l'instance HTTP : injection automatique du header `Authorization`, tentative de rafraîchissement de session sur une réponse 401 (hors routes `/auth/*` publiques) avec rejeu de la requête d'origine, et affichage de toasts d'erreur centralisés par code HTTP. Un flag `_silent` permet de désactiver ce toast pour un appel donné. En pratique, de nombreuses pages (Dashboard, Rechargement, Header, Team, Profile, Settings...) appellent directement l'instance `api` plutôt que de passer par un service dédié dans `src/api/`, ce qui relâche la séparation service/UI par endroits.

3.1 Rôle

Inscription, connexion avec 2FA optionnelle, confirmation d'email, mot de passe oublié, acceptation d'invitation d'équipe.

3.2 Fichiers réellement actifs

- **Login.tsx (route /login)** !' formulaire email/mot de passe, bloc conditionnel de saisie du code 2FA
- **Register.tsx + RegisterForm.tsx** !' inscription avec react-hook-form + zodResolver (règles mot de passe : 8 caractères min., majuscule, chiffre, caractère spécial)
- **ProtectedRoute.tsx / RoleGuard.tsx** !' gardes d'authentification et de rôle (config centralisée dans src/config/roles.ts)
- **Security.tsx (2FA)** !' génération de QR code via la librairie qrcode (QRCode.toCanvas) pour l'activation TOTP, alternative SMS, codes de secours

& Code mort identifié dans ce module

Les pages de connexion non prises en compte de Login.tsx et useRegister.ts (chaque logique de RegisterForm.tsx) sont présentes dans le dépôt mais jamais implémentées par le mainteneur — voir chapitre 14 pour la liste complète

Tests e2e (e2e/auth.spec.ts, 7 scénarios) : affichage du formulaire, connexion valide !' redirection dashboard, mauvais mot de passe !' message d'erreur, mot de passe oublié !' reset, route protégée sans session !' redirection login.

4 — Dashboard

Page d'accueil post-connexion (src/pages/Dashboard.tsx, 882 lignes, la page la plus dense en logique après Automations/Settings). Deux modes basculables par double-clic sur "Tableau de bord" dans la barre latérale : vue KPIs/Analytics (par défaut) et vue opérationnelle (campagnes récentes, automatisations actives, logs d'audit, solde crédit).

4.1 Sources de données

useAppMetrics() (solde + total contacts), useCampaignStore (liste de campagnes), puis des appels directs à /analytics/overview, /automations, /audit-logs?limit=5, /account/balance via des useEffect indépendants — sans mutualisation ni cache entre les appels.

&™ Fait notable — graphiques

La librairie recharts est déclarée en dépendance mais jamais importée dans le code. Les graphiques (courbe d'évolution, donut par canal) sont dessinés en SVG fait-main (polyline, conic-gradient CSS) — un choix qui fonctionne mais duplique une logique que recharts aurait fournie nativement.

5 — Campagnes — liste, wizard, éditeurs, rapport

5.1 Liste et suppression

CampaignListDashboard.tsx : filtres (statut, canal, recherche, tri), regroupement campagnes classiques vs automatisées, suppression avec confirmation via @radix-ui/react-dialog — seul usage de Radix Dialog dans tout le projet.

5.2 Wizard de création (4 étapes)

- **CampaignChannelStep** !' choix SMS ou Email (pas de WhatsApp à ce niveau, contrairement aux automatisations)
- **CampaignContentStep** !' EmailEditor (blocs drag & drop, A/B testing complet sujet/preheader/blocs séparés, import HTML) ou SMSEditor (compteur de caractères, calcul de segments, bloc STOP obligatoire — règle métier RG-22)
- **CampaignAudienceStep** !' ciblage par segment
- **CampaignScheduleStep** !' planification, avec BestSendTimePicker et CancellationControl

5.3 Rapport de campagne

CampaignReport.tsx (route /campaigns/:id/report) : KPIs d'envoi, tableau des contacts ayant ouvert/cliqué, heatmap de zones de clic, export CSV client. Utilise un style Tailwind "Material 3"-like distinct du reste de l'application en CSS custom — incohérence de design system entre pages, à noter.

& Dette technique majeure identifiée

Le dossier src/features/campaigns contient src/features/campaigns/list, src/features/campaigns/wizard et src/features/campaigns/report qui contiennent une seconde implémentation complète et autonome du wizard de campagne, non référencée par aucune route. Deux implémentations parallèles coexistent dans le dépôt, une seule est branchée au routing react.

6.1 Liste et fiche contact

Contacts.tsx (320 lignes) : liste + modale d'ajout avec validation téléphone en temps réel (debounce 600ms).
ContactTable.tsx : tableau virtualisé via @tanstack/react-virtual (performant sur de grandes bases).
ContactDetail.tsx (route /contacts/:id) : historique d'activité (Timeline.tsx).

6.2 Import CSV/Excel

ImportModal.tsx : flux multi-étapes (upload !' mapping de colonnes !' aperçu !' progression !' rapport), parsing via papaparse (useCsvParser.ts), transitions animées avec framer-motion. Appelle directement l'API (pas via le service contacts.ts) : upload !' démarrage job chunké !' polling de statut !' complétion, avec statistiques live (succès, doublons, téléphones invalides).

6.3 Segments — fonctionnalité orpheline dans le routing

& Fait important à documenter

Deux implémentations dédiées de la gestion de segments existent (pages/segments.tsx, 411 lignes, build et build:prod complet : components/segments/segmentBuilder.tsx) mais aucune n'est reliée à une route. La route dédiée segments dans main.tsx pointe en réalité vers le composant Contacts. La fonctionnalité segments, bien qu'elle soit accessible aux utilisateurs via celle intégrée dans le sous-composant SegmentDetail de ContactTable.tsx, n'est apparemment pas activée dans les tests e2e car elle n'est connectée par rien à une action.

Tests e2e (segments.spec.ts) confirment ce comportement : ils naviguent vers /contacts, pas /segments, pour vérifier le bouton "créer un segment".

7 — Automations

Automations.tsx (1565 lignes — la page la plus volumineuse du projet) : liste des automatisations, création via formulaire ou via des templates de workflow pré-remplis (WORKFLOW_TEMPLATES). L'éditeur visuel CanvasEditor.tsx est un canvas drag & drop fait main (positions x/y en pixels, gestion souris manuelle) — aucune librairie de diagramme (type React Flow) ni @dnd-kit n'est utilisée malgré sa présence en dépendance.

Fait notable : contrairement au wizard de campagnes (SMS/Email uniquement), le type DraftAutomation.channel inclut Email, SMS et WhatsApp — le canal WhatsApp n'apparaît donc que dans les automatisations et la légende du dashboard, jamais dans la création de campagne classique.

Analytics.tsx (609 lignes) : sélecteur de période, 5 KPIs (envoyés, taux d'ouverture, taux de clic, rebonds, désabonnements), graphique d'évolution SVG fait-main, top campagnes, export CSV côté client. Consomme le même endpoint GET /analytics/overview que le Dashboard, de façon totalement indépendante (pas de déduplication de requête, cohérent avec l'absence de React Query dans le projet).

9 — Rechargement / Billing

11

Rechargement.tsx (1133 lignes, page la plus dense en logique métier du frontend) : achat de crédits via Mobile Money (Wave, Orange Money, MTN MoMo, Moov) ou carte Visa. Validation par opérateur (préfixes téléphoniques CI, montants min/max), OTP obligatoire pour Orange Money, polling de statut de paiement toutes les 3 secondes pendant 2 minutes maximum. Un CustomEvent('novasms:balance-refresh') diffusé sur window synchronise le solde affiché entre Header, Sidebar et Dashboard après un rechargement réussi — pattern d'event bus DOM natif utilisé faute de store de solde partagé.

Route accessible au seul rôle Admin (RoleGuard roles={['Admin']}).

Page	Fichier	Rôle
Profil	Profile.tsx (462 l.)	Édition entreprise/pays, mot de passe
Sécurité	Security.tsx (579 l.)	2FA TOTP/SMS, codes de secours
Paramètres	Settings.tsx (1511 l.)	Préférences, langue, clés API, webhooks
Équipe	Team.tsx (799 l.)	Invitations, rôles, consommation crédit par membre
Journal d'audit	AuditLogs.tsx (401 l.)	Historique paginé des actions du compte
Intégrations	Integrations.tsx (239 l.)	Santé des fournisseurs SMS/Email connectés
Développeurs	Developers.tsx (1072 l.)	Clés API (création, révocation, stats)

Ces pages sont réservées à des rôles précis via RoleGuard (ex. Team/Developers : Admin seul ; AuditLogs : Admin+Analyst), reflétant côté frontend le RBAC implémenté côté backend (Doc 2, chapitre 3).

11 — Modèle de données — relations clés côté produit

13

Le détail exhaustif des 26 modèles Prisma est présenté dans le Doc 2 (chapitre 2). Ce chapitre relie les relations les plus structurantes du modèle de données aux écrans du frontend qui les exploitent.

Relation (BDD)	Cardinalité	Écran frontend concerné
Account !' User	1 !' N	Team.tsx (gestion d'équipe)
Account !' Contact	1 !' N	Contacts.tsx, ContactTable.tsx
Contact !' Send !• Campaign	N !' N (via Send)	CampaignReport.tsx, Analytics.tsx
Campaign !' CampaignImage	1 !' N	EmailEditor.tsx (upload d'images)
Campaign !' ABTestResult	1 !' 1	ABReport.tsx
Segment !' Contact	1 !' N (critères dynamiques)	ContactTable.tsx / SegmentDetail
Automation !' WorkflowExecution	1 !' N	Automations.tsx (historique d'exécution)
Account !' Transaction / MnhileMonevTransaction	1 !' N	Rechargement.tsx
Account !' ApiKey !' ApiKeyLog	1 !' N !' N	Developers.tsx
Account !' AuditLog	1 !' N	AuditLogs.tsx

Point d'architecture notable : il n'existe pas de package de types partagés généré automatiquement entre le backend (types Prisma) et le frontend (src/types/). Les types frontend sont maintenus manuellement en miroir des DTOs backend, ce qui a directement contribué à un bug de build identifié dans le Doc 4 (type AutomationTrigger désynchronisé après l'ajout du déclencheur "anniversaire" côté backend).

Reconstitution du parcours nominal d'un nouveau client, du premier contact avec la plateforme jusqu'à l'envoi de sa première campagne, tel qu'implémenté par les écrans et endpoints décrits dans ce document et le Doc 2.

- 1 Inscription (Register.tsx)**
POST /auth/register — création du compte (Account), email de vérification envoyé via MailModule
- 2 Vérification d'email**
GET /auth/verify-email/:token — activation du compte (emailVerified = true)
- 3 Connexion + onboarding**
POST /auth/login puis /auth/onboarding/complete — checklist guidée (driver.js) : profil, import, campagne, envoi
- 4 Import de contacts**
ImportModal.tsx !' upload chunké !' POST /contacts/import/* — job BullMQ, rapport de synthèse
- 5 Création d'un segment (optionnel)**
Ciblage dynamique via SegmentDetail (ContactTable) ou tags simples
- 6 Création de campagne**
CampaignWizard (4 étapes) !' POST /campaigns puis /campaigns/:id/save-draft
- 7 Vérification du coût**
POST /campaigns/sms/calculate-cost — déduction anticipée du solde de crédits affichée
- 8 Envoi**
POST /campaigns/:id/send — mise en file BullMQ (campaign-dispatch), traitement par lots de 500 contacts
- 9 Suivi**
CampaignReport.tsx / Analytics.tsx — ouvertures et clics remontés en temps réel via /track/open et /track/click

13 — Parcours utilisateur — recharge de crédit Mobile Money

13

1

Accès à la page Rechargement

Route réservée au rôle Admin (RoleGuard)

2

Choix de l'opérateur

Wave / Orange Money / MTN MoMo / Moov — règles de préfixe et de montant appliquées en direct (OPERATOR_RULES)

3

Saisie du numéro + montant

Validation locale avant soumission (ex. Wave : préfixes 01/05/07/27, 500–500 000 FCFA)

4

Initiation du paiement

POST /mobile-money/initiate — création d'une MobileMoneyTransaction en statut Pending

5

OTP (si Orange Money)

Saisie du code envoyé par l'opérateur, requis avant confirmation

6

Polling de statut

GET /mobile-money/:id/status toutes les 3s pendant 2 minutes maximum côté client

7

Confirmation

Statut Validated !' CustomEvent novasms:balance-refresh diffusé !' solde mis à jour partout (Header, Sidebar, Dashboard)

8

Reçu

GET /mobile-money/:id/receipt — PDF généré via pdfkit, téléchargeable

14 — Incohérences et dette technique frontend identifiées

10

Cette synthèse recense, tel quel, l'écart entre le code présent dans le dépôt et le code réellement actif — un exercice volontairement transparent, utile pour prioriser les prochaines itérations et pour anticiper les questions du jury sur la qualité du code livré.

14.1 Fichiers/dossiers jamais importés par le routing ou un autre module actif

- **src/App.tsx** !' retourne null, vestige du template Vite initial
- **src/features/auth/pages/LoginPage.tsx** !' doublon non utilisé de Login.tsx
- **src/features/auth/hooks/useRegister.ts** !' doublon logique de RegisterForm.tsx
- **src/lib/validation.ts** !' schéma zod mort, testé uniquement par son propre test (ne teste donc pas le schéma réellement utilisé en production)
- **src/pages/Segments.tsx + src/components/segments/SegmentBuilder.tsx** !' non reliés au routing (voir chapitre 6.3)
- **src/features/campaigns/ (dossier entier)** !' seconde implémentation abandonnée du wizard de campagne
- **src/components/campaigns/TemplateLibrary.tsx** et **src/api/templatesApi.ts** !' remplacés respectivement par TemplatePreviewLibrary.tsx et une constante locale EMAIL_TEMPLATES
- **src/components/WelcomeChecklist.tsx + src/hooks/useOnboardingChecklist.ts** !' non utilisés, le Dashboard réimplémente sa propre checklist inline

14.2 Dépendances déclarées mais jamais importées dans src/

recharts, @dnd-kit/core, @dnd-kit/sortable, @dnd-kit/utilities, react-email-editor, class-variance-authority, tailwind-merge, clsx, date-fns, date-fns-tz, @radix-ui/react-dropdown-menu, @radix-ui/react-toast.

14.3 Incohérences fonctionnelles

- **Validation de formulaires non homogène** !' react-hook-form + zod uniquement sur l'inscription ; tous les autres formulaires (Login, Contacts, Rechargement, Security, Profile, Settings, Team) sont contrôlés manuellement
- **Pas de dark mode** !' aucune configuration Tailwind darkMode ni media query prefers-color-scheme
- **Accessibilité partielle** !' efforts ciblés (AppLayout, Header, ContactTable) mais absents sur la majorité des formulaires métier
- **i18n non déployée** !' infrastructure i18next initialisée globalement mais utilisée sur seulement 2 clés de traduction ; 100% des pages métier sont codées en dur en français

Q — Cette dette technique remet-elle en cause la solidité du produit livré ?

Non : l'ensemble des parcours utilisateur nominaux fonctionne (voir Doc 4, résultats des 45 scénarios end-to-end, tous passants). La dette identifiée ici concerne des itérations de développement où une seconde approche a été tentée puis abandonnée sans nettoyage du code mort — un phénomène courant en développement rapide solo sous contrainte de temps. La documenter explicitement, plutôt que de la dissimuler, est en soi une démarche de rigueur attendue dans un mémoire technique : elle montre une capacité d'audit critique du propre travail du candidat.